Employee Management System

Student ID: M01044595

Name: Abubakar

Student ID: M01041984

Name: Alishba Abid

Student ID: M01048578
Name: Farhan Saleem
Student ID: M01048578

Name: Ahmed Hanjala Remon
Student ID: M01042412
Name: Syed Safiuallah Inayat

Student ID: M01044448

1. Introduction

This project outlines the design and implementation of an Employee Management System written in C# .NET as a console-based application. The system is designed to manage employee records and associated leave requests in an organised and efficient manner. It provides core functionality including adding employee profiles, searching employee data, recording leave requests, and persisting stored information. The project addresses a realistic organisational need where effective employee tracking and data retrieval are critical to operational efficiency.

The central concern of this project is the selection and implementation of an appropriate data structure to facilitate efficient search and data management. Rather than relying on built-in collection libraries, the system is built on a custom-designed Binary Search Tree, demonstrating a thorough understanding of algorithmic design and complexity analysis. This approach is consistent with the coursework requirement of prioritising the implementation and performance analysis of a custom data structure (Košťál, 2023).

The system is developed following object-oriented design principles to ensure maintainability and transparency. The data structure, employees, and leave records are represented in separate classes, making the system modular and easy to extend. Additionally, file management and SQL database scripting are included to support persistent data storage, further enhancing the practicality of the system.

This report contains four main sections. The Design section discusses the system architecture, justifies the chosen data structure, and presents the key algorithms using pseudocode. The Testing section explains the testing methodology and provides a structured table of test cases to validate the system's functionality. The Conclusion section summarises the work completed, discusses limitations, and offers a critical reflection on the development process along with suggestions for future improvements. Finally, the References section lists all sources cited in Harvard style.

2. Design

The Employee Management System design is centred on efficient data storage and retrieval, combined with a clear and modular architecture. The system applies object-oriented principles, with each component encapsulated in a dedicated class. The main classes are the Employee class, which stores employee information such as ID, name, department, and role, and the LeaveRecord class, which holds leave application details. These classes represent real-world entities, ensuring the system structure closely mirrors practical use cases.

At the core of the system is the Binary Search Tree (BST) data structure, which serves as the central mechanism for storing employee records. Each BST node contains an Employee object and references to left and right child nodes. The tree is ordered by employee ID, with all nodes in the left subtree having smaller IDs and those in the right subtree having larger IDs. This structural property enables efficient search, insertion, deletion, and traversal operations (Jangra et al., 2023).

The Binary Search Tree was chosen based on its performance characteristics and suitability for the problem domain. Efficient searching is a primary requirement of the system, as users frequently need to locate employee records. Compared to linear data structures such as arrays or linked lists, which require O(n) time for search operations, a BST offers an average-case complexity of O(log n). This represents a significant performance improvement, particularly as the dataset grows. Furthermore, the BST maintains data in sorted order, enabling structured traversal and streamlined data management.

Alternative data structures were evaluated during the design phase. Arrays were rejected due to their fixed size and inefficient insertion operations, which may require shifting elements. Linked lists were considered unsuitable because of their linear search complexity, despite offering dynamic sizing. Hash tables were proposed but ultimately not selected, as they do not inherently maintain sorted order and would not clearly demonstrate the algorithm design and traversal logic required by the coursework specification (Panjuta and Jabbarzadeh, 2024). Consequently, the Binary Search Tree was identified as the optimal choice in terms of efficiency, readability, and educational value.

The insertion algorithm in the BST operates recursively. When a new employee is added, the algorithm compares the employee's ID with the current node's ID. If the new ID is smaller, the algorithm traverses the left subtree; if larger, it traverses the right subtree. This process repeats until an empty position is found, where the new node is then inserted. The pseudocode for this operation is as follows:

```
FUNCTION Insert(node, employee)
   IF node is NULL
      RETURN new node with employee
   IF employee.ID < node.employee.ID
      node.left = Insert(node.left, employee)
   ELSE IF employee.ID > node.employee.ID
      node.right = Insert(node.right, employee)
   RETURN node
```

The search algorithm is also implemented recursively with similar traversal logic. The algorithm compares the target ID with the current node's ID and moves to either the left or

right subtree accordingly (Golmohammadi, Zhang and Arcuri, 2023). This approach is efficient as only a subset of nodes is explored. The pseudocode is as follows:

```
FUNCTION Search(node, id)
   IF node is NULL OR node.employee.ID = id
      RETURN node
   IF id < node.employee.ID
      RETURN Search(node.left, id)
   ELSE
      RETURN Search(node.right, id)
```

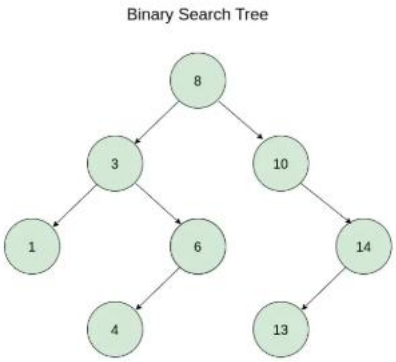

Figure 1: BST (Geeksforgeeks, 2024)

The deletion algorithm is the most complex of the three core BST operations. When removing a node, three cases must be handled: the node has no children (leaf node), the node has one child, or the node has two children. In the first two cases, the node is simply removed or replaced by its single child. In the third case, the node's value is replaced with its in-order successor (the smallest node in the right subtree), and the successor is then recursively deleted. This ensures the BST ordering property is preserved (Jamro, 2024). The pseudocode is as follows:

```
FUNCTION Delete(node, id)
   IF node is NULL
      RETURN NULL
   IF id < node.employee.ID
      node.left = Delete(node.left, id)
   ELSE IF id > node.employee.ID
      node.right = Delete(node.right, id)
   ELSE
      // Node found - handle three cases
      IF node.left is NULL
         RETURN node.right
      ELSE IF node.right is NULL
         RETURN node.left
      ELSE
         // Two children: find in-order successor
         successor = FindMin(node.right)
```

```
            node.employee = successor.employee
            node.right = Delete(node.right, successor.employee.ID)
      RETURN node

   FUNCTION FindMin(node)
      WHILE node.left is NOT NULL
         node = node.left
      RETURN node
```

The delete operation has a time complexity of O(log n) in the average case and O(n) in the worst case, consistent with the insert and search operations. The FindMin helper function traverses left until it reaches the leftmost node, which runs in O(h) time where h is the height of the subtree.

In addition to ID-based search, the system supports search by employee name. Since the BST is not ordered by name, this operation requires a full traversal of the tree. An in-order traversal visits all nodes and checks each for a match against the search query. Although this operation has a time complexity of O(n), it is necessary to provide flexible search functionality.

The system also incorporates leave record management. Each Employee object contains a list of leave records, allowing multiple leave entries per employee. When a leave record is added, the system first performs a search operation to locate the relevant employee, then appends the leave record. This demonstrates the integration of multiple data structures within the system (Sigüenza Vilella, 2025).

A critical aspect of the design is the time complexity of the core operations. On average, insertion, search, and deletion in a BST all operate at O(log n) when the tree is balanced. However, in the worst case, when the tree becomes skewed due to sequential insertions of sorted data, the complexity degrades to O(n). This limitation is acknowledged and discussed further in the conclusion section.

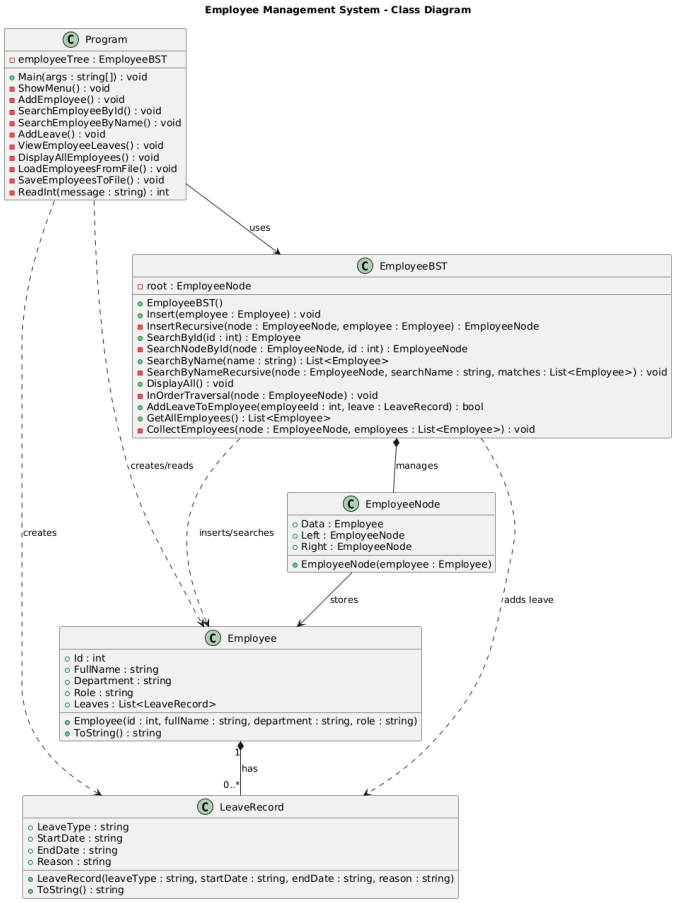

Figure 2: Class Diagram

Overall, the system design achieves a strong balance of performance, readability, and maintainability. The use of a Binary Search Tree not only ensures efficient data handling but also satisfies the coursework requirement of implementing a custom data structure and analysing its behaviour (Panjuta and Jabbarzadeh, 2024).

3. Testing

Testing is a critical component of the software development process, as it confirms the system's functionality and validates that it meets the specified requirements. A structured testing methodology was employed in this project, combining unit testing using MSTest with manual functional validation. The aim was to ensure that every feature operates as intended and that the system behaves reliably under various conditions (Griffiths, 2022).

Testing commenced at the unit level, validating individual components in isolation. Core functions including employee insertion, searching, deletion, and leave management were tested separately to ensure correctness. This approach facilitated the isolation of potential issues and enabled targeted debugging (Akella et al., 2022). Integration testing followed, verifying that the various components interact correctly when combined.

Search functionality testing was a major priority, given that efficient searching is the primary purpose of the data structure. Multiple scenarios were tested, including searching for existing employees, searching for non-existent employees, and performing partial name searches. The results confirmed that the BST implementation delivers accurate and efficient search outcomes (Jamro, 2024).

Input validation testing was also conducted to ensure the system handles erroneous inputs gracefully, such as entering non-numeric values for employee IDs. This contributes to the robustness and stability of the system (Shoykulov, 2024). File handling was also tested by loading employee data from a file and saving data to a file, verifying that data is correctly parsed and stored. This aspect is essential for demonstrating data persistence and fulfilling the coursework requirements.

The table below summarises the test cases used during testing:

Test Case ID	Description	Input	Expected Output	Result
TC01	Add valid employee	ID=101	Employee added successfully	Pass
TC02	Add duplicate employee	ID=101	Error message displayed	Pass
TC03	Search by valid ID	101	Employee details shown	Pass
TC04	Search invalid ID	999	Employee not found	Pass
TC05	Search by name	"Ali"	Matching employees listed	Pass
TC06	Add leave record	Valid data	Leave added successfully	Pass
TC07	View leave records	Employee with leaves	Leave details displayed	Pass
TC08	Load from file	Valid file	Employees loaded	Pass
TC09	Save to file	output.txt	File created successfully	Pass
TC10	Invalid input	Text instead of number	Error handled correctly	Pass

TC11	Delete existing employee	ID=103	Employee removed from BST	Pass
TC12	Delete non-existent employee	ID=999	Employee not found message	Pass

The testing results confirm that the system operates reliably across all core functionalities. Any minor issues identified during development were addressed through iterative testing. Overall, the testing process demonstrates that the system meets its functional requirements and performs as expected.

4. Conclusion

This project has successfully achieved its objective of designing and implementing an Employee Management System in C# .NET. The system provides a functional solution for managing employee records and leave applications, demonstrating the practical application of data structures and algorithms. A Binary Search Tree was implemented as the core data structure, enabling efficient storage and retrieval of data in accordance with the coursework requirements.

The project demonstrates competence in key programming concepts including object-oriented design, recursion, and complexity analysis. The system is modular, maintainable, and supports essential operations such as insertion, searching, deletion, and data persistence. Additionally, file handling and an SQL database script are included to enhance the system's practicality.

Nevertheless, several limitations were identified during the development process. The most significant weakness is that the Binary Search Tree may become unbalanced, leading to degraded performance in the worst case. This occurs because no self-balancing mechanism, such as AVL or Red-Black Trees, is implemented. Additionally, the console-based interface limits user interaction and does not provide the usability expected of modern applications. A further constraint is the absence of live database integration, which restricts the system's scalability and real-world applicability.

Critical reflection on the development process suggests that scalability and performance optimisation should have been considered more carefully during the design phase. While the BST met the coursework requirements, a more advanced data structure could have been employed to guarantee consistent performance. Furthermore, a more structured development process, such as incremental prototyping or automated testing from the outset, would have improved efficiency and reduced development time.

Several enhancements could be made to the system in future. Implementing a self-balancing tree would ensure consistent performance across all scenarios. Developing a graphical user interface would greatly improve usability and accessibility. Integrating the system with a relational database using SQL would provide more robust data management capabilities. Additionally, expanding the automated testing suite using MSTest would further enhance reliability and maintainability.

In conclusion, this project serves as a solid demonstration of data structure implementation and algorithm analysis within a practical application. It highlights both the strengths and limitations of the chosen approach and provides a strong foundation for future development.

References

Akella, R., Tamirisa, A.K., Kunani, S.K. and Muthiyalu, B.G., 2022. Enterprise Application Development with C# 10 and .NET 6: Become a professional .NET developer by learning expert techniques for building scalable applications. Packt Publishing Ltd.

Golmohammadi, A., Zhang, M. and Arcuri, A., 2023. .NET/C# instrumentation for search-based software testing. Software Quality Journal, 31(4), pp.1439–1465.

Griffiths, I., 2022. Programming C# 10: Build Cloud, Web, and Desktop Applications. O'Reilly Media, Inc.

Jamro, M., 2024. C# Data Structures and Algorithms: Harness the power of C# to build a diverse range of efficient applications. Packt Publishing Ltd.

Jangra, S., Singh, G., Mantri, A., Angra, S. and Sharma, B., 2023. Interactivity development using Unity 3D software and C# programming. In 2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT) (pp. 1–6). IEEE.

Košťál, I., 2023. Execution Efficiency of a Sequential, Multi-Threaded, and a Parallel Search for a Structured Data in a Symbol Table Implemented in a Binary Search Tree, an Array, and in a Linked List. In International Conference on Global Research and Education (pp. 316–332). Springer Nature Switzerland.

Microsoft, 2024. C# Documentation. Available at: https://learn.microsoft.com (Accessed: 2026).

Panjuta, D. and Jabbarzadeh, J., 2024. Learning C# Through Small Projects. Springer.

Shoykulov, S., 2024. Analysis and optimization of graphics programming in C# using Unity. Science and Innovation, 3(A10), pp.69–75.

Sigüenza Vilella, A., 2025. Design and implementation of a graphic console for a home automation system focused on security.